Explainability for Systems: Agua Revisited

Coen Adler *Sihat Afnan* *Michael Mulder*

Abstract

One major barrier to the adoption of learning-enabled controllers for systems and networking domains is their lack of explainability. Agua [13] pioneered a concept-based explainability framework designed to address this limitation. In this paper, we propose three extensions to Agua. All three approaches yield surrogate models with high fidelity to their respective controllers (>96%), but at the cost of added complexity. Each extension differs in how closely it adheres to Agua's architecture and occupies a distinct point on the spectrum from research prototype to deployment-ready system.

1 Introduction

Machine learning (ML) has become deeply integrated into modern systems, enabling controllers that dynamically adapt to complex environments such as video streaming, congestion control, and network security. These learning enabled controllers consistently outperform traditional, rule based systems by discovering optimized control policies directly from data [6, 10, 20]. However, their opaque nature makes them difficult to interpret, debug, and trust in production settings. In contrast to conventional systems governed by explicit logic, ML controllers encode decision rules implicitly in high dimensional embeddings and nonlinear mappings. As a result, system operators often struggle to understand why a controller selects a given action, or how its internal representations correspond to meaningful system states.

Explainable AI (XAI) methods have made considerable progress toward interpreting black box models. Feature attribution approaches such as SHAP [9] and LIME [15] estimate each input feature's contribution to a model's output, providing local, instance level explanations. These approaches have proven valuable for domains like medical diagnosis or tabular data analysis, where individual features correspond to interpretable quantities. Yet, their direct application to systems controllers is limited. Inputs in ML forsystems tasks are often temporally correlated, heterogeneous, and high dimensional for instance, time series of throughput, latency, and buffer occupancy. Featurelevel attributions fail to capture emergent behaviors that arise from interactions among these signals, offering little insight into higher level abstractions such as buffer underflow risk or anticipation of congestion.

The recently proposed Agua framework [13] addresses this limitation by introducing concept based explanations for learning enabled systems. Instead of relying solely on feature importances, Agua builds a surrogate model that learns a structured, human aligned concept space. Its pipeline consists of two stages: a concept mapping function, which projects a controller's internal embeddings into concept activations (e.g., high buffer volatility), and an output mapping function, which linearly combines these concepts to reproduce the controller's decisions. This design enables operators to reason about system behaviors at a conceptual level, bridging the gap between numerical features and human understandable explanations. Agua thus demonstrates that concept bottlenecks, previously explored mainly in computer vision [4, 7, 8] can be adapted to complex, temporal, and heterogeneous systems settings.

In this project, we build upon the key ideas of Agua and aim to explore several extensions that could improve the fidelity and interpretability of concept based explanations for learning enabled systems. Specifically, we explore three directions:

- **Concept Embedding Model:** We enrich the latent dimension of the surrogate by moving from scalar concepts to concept "embeddings".
- **Nonlinear Output Mapping:** We replace Agua's linear output mapping with a lightweight neural network.
- **LLM-Generated Annotations for Learned Representations:** We train a convolutional neural network via imitation learning, then use an LLM to annotate the learned representations.

2 Motivation

Machine learning plays a large role in system controllers, where controllers make decisions based on network traffic that influence user experience. While machine learning approaches have shown to outperform traditional models [6, 10, 20], they come at the cost of interpretability. These black box models decrease operator trust and usability in these systems. We see Agua—and concept-based explanations more generally—as the best surrogates for making sense of complex deep-learning models.

3 Related Work

3.1 XAI methods

Deep learning models are black boxes in that their set of computations are inherently not human interpretable. Within the field of Explainable AI (XAI), a series of explainable methods exist to help uncover how a model converts its inputs into a final prediction. There are a variety of methods capable of explaining these black boxes, but most tend to fall into

one of two categories: perturbation and propagation-based methods [2].

Perturbation methods treat the original model as a black-box that can be queried but no information can be retrieved besides the models output given any input. The most popular perturbation method is SHAP values [9] which use a game-theoretic approach to identify the attribution of each feature to a model's output. Another approach is LIME [15], which trains a local interpretable surrogate model for the original black box.

While perturbation methods treat the original model as a black-box, propagation-based methods assume that additional information about the model are available. A common approach in propagation-based methods like Integrated Gradients [19] and Grad-CAM [18], is to use model gradients with respect to a specific target variable to explain the features.

3.2 Concept Bottleneck Models

Attempting to explain large DL models has its limitations with explanation methods often disagreeing with one another [16] and many explanations can often be computationally expensive to produce [2]. Rather than explaining a black box model, training an inherently interpretable model like a small decision tree or linear model can satisfy interpretability requirements. However, many of these interpretable models are also limited in their expressivity. Another approach is to incorporate human-interpretable concepts directly into the model's latent space. Concept bottleneck models [8] do this by splitting a standard model into two sub-models (concept mapping and output mapping model).

The initial concept mapping model learns to predict a set of predefined concepts given an initial input, while the output mapping model uses the concepts to predict a final output. This two-tiered system allows for practitioners to get a better understanding of how the model has come to the final prediction. Because the model produces human interpretable concepts at an intermediary layer, humans can easily debug or identify a model's chain of thought. Typically, the output mapping model is a single layer neural network or similarly interpretable model allowing further attribution of concepts to the model's output.

3.2.1 Concept Embedding Models

Concept Embedding Models [3] or CEMs are a recent improvement to the CBM model. In addition to producing scalar concept activations, CEMs learn a multi-dimensional concept embedding that can contain more information than a scalar. This improvement allows CEMs to produce more accurate outputs due to a higher model capacity that is no longer limited by the concept bottleneck layer in a CBM.

In a CEM, each concept is represented with both a positive and a negative concept embedding. Using the learned embeddings, the scalar concepts can be generated via a linear layer and are represented as the concept probability similarly to a CBM. The output mapping function takes in an aggregate concept embedding that is produced as a convex combination of both the positive and negative embeddings where they are weighted by the probability the concept is activated.

Interventions remain an essential capability of the CEM model. Because the model produces a positive and negative concept embedding, an expert can still intervene on the scalar concept without the understanding of the positive and negative embeddings. The updated scalar concept activation will modify the final concept embedding as the weights of the linear combination are dependent on the intervened concept.

3.2.2 Unsupervised Learning of Concepts

Concept bottleneck models are often limited by data and concept availability [3, 22] Producing concepts often require human annotators that are hard to scale for more complex problems. An unsupervised approach by Schrodi et al. [17] utilizes a sparse layer as the concept bottleneck in an attempt to align the unsupervised sparse features with human interpretable concepts. A recent solution to this is annotations using LLMs [21], where an LLM is given the model's input and produces the target concepts.

4 Design

We explored three extensions to Agua. First, we examined transitioning Agua from a concept bottleneck model to a concept embedding model (Section 4.1). Second, we investigated replacing Agua's linear output mapping with a small multilayer perceptron (Section 4.2). Finally, we experimented with using an LLM to annotate the filters learned by a lightweight deep-learning model (Section 4.3). Each extension is self-contained in its subsection.

4.1 Agua as a Concept Embedding Model

As explained above, Agua is designed following the Concept Bottleneck Model (CBM) architecture, where a concept mapping function learns predefined concepts and an output mapping function uses the concepts to predict the original model's outputs. The downstream performance of CBMs are often limited by the concept layer, where the intermediary concepts are not informative enough to predict the output. This can lead to data leakage, where the concept activations are polysemantic, meaning the activations may contain more information than just the concept itself. While information leakage through the concept layer may improve the model's output performance, it may come at the cost of worse concept predictions and explanations.

Concept Embedding Models [3] build off of CBMs by purposefully utilizing the concept of information leakage while

maintaining the explainable concepts. Agua can be easily implemented as a CEM rather than a CBM. The concept mapping function will generate both the concept embeddings and the desired scalar concepts. Meanwhile, the output mapping function takes in the concept embeddings (rather than scalar concepts) to predict the original model's outputs. A key difference is how explanations from the CEM w.r.t. the scalar concepts are generated. In CBMs, the explanations are simply the magnitude of the concept activation and the output mapping weights. Because the output function in a CEM uses the concept embeddings as the input, we have to aggregate the attribution from each neuron in a single embedding and use the sum of the attributions as the concept's total attribution.

A more complex solution is using permutation-based explainability methods like LIME and SHAP w.r.t. the scalar concepts. The final concept embeddings that are used in the output mapping function are a convex combination of a positive and negative concept embedding. Therefore, by intervening and modifying the scalar concept activations, the final concept embeddings will also be updated. Using this ability, we can apply permutation-based explainability methods to directly retrieve the output mapping functions attributions with respect to the scalar concepts rather than non-interpretable individual components of the concept embedding.

4.2 Non-Linear Output Mapper

Agua models the relationship between concepts and controller actions using a sparse linear classifier based on ElasticNet regularization. While this linear head is easy to interpret—each weight directly reflects the contribution of a concept to an action, it also restricts the surrogate model to *additive* concept–action relationships. In practice, system controllers often rely on richer, non-linear interactions among concepts (e.g., jointly high congestion anticipation *and* rapid buffer depletion), which a purely linear mapper may fail to capture.

To explore whether a more expressive mapping can improve fidelity, we design a lightweight neural network as a drop-in replacement for Agua's linear output mapper. The goal is to maintain simplicity and interpretability while enabling the model to learn non-linear relationships between the concept activations and the controller's actions.

Interpretability Considerations. Replacing the linear model removes the direct interpretability of weight-based concept attributions. In future work, this limitation can be addressed using gradient-based saliency methods (e.g., Integrated Gradients, SHAP values) to recover per-concept importance. In this study, our primary goal is to investigate whether introducing mild non-linearity can improve Agua's predictive fidelity without excessively compromising interpretability.

4.3 An Agua-Inspired Surrogate for Congestion Control

Agua relies on a LLM to conjecture base concepts as well as to transcribe multivariate time series input into natural language descriptions. In this section, we explore instead using the LLM to generate post-hoc explanations for filters learned by a lightweight 1D convolutional neural network. We followed the pipeline:

1. Train a small 1D convolutional neural network on the controller's inputs x and outputs y
2. Use a LLM to describe the learned representations. The LLM relies on the descriptions generated for one layer to help interpret the next.

So while Agua relies on the LLM to restrict its representational capacity, thereby offering greater interpretability, our approach allows our learner to find more complex representations, thus offering greater fidelity to the original controller. Importantly, we show that our flexible learner is interpretable despite being another neural network.

Architecture: Training one neural network to interpret another might seem like the first step in an infinite regress. How might the surrogate network be made interpretable? We leveraged two insights:

1. The first layer of a neural network is interpretable, as the inputs to this layer are the inputs to the network itself.
2. CNNs are well equipped to detect 1D "shapelets" in multivariate time series data. Indeed, convolutional neural networks achieved SOTA performance on time series classification circa 2019 [5].

Such considerations led us to opt for a 1D CNN with a wide first layer. Our network has four main components: 1) a parallel depthwise convolutional layer, 2) a pointwise (1 x 1) convolutional layer, 3) a global average pooling layer, and 4) an output layer (see the complete architecture in Figure 1):

0. Input: We evaluate our approach on Aurora, a deep RL controller for congestion control. Aurora takes as input statistics about the client's latency, throughput, and loss, over the previous 10 time steps and outputs a discretized adjustment to the current data transmission rate. Seven adjustments, or actions, were available to Aurora during training: -50%, -33%, -10%, No change, +10%, +33%, +100%. Aurora, however, never selects the last action, +100%, so the size of the action space is effectively 6.

Matching Agua, our surrogate CNN was trained on just a subset of Aurora's input statistics: loss ratio, latency ratio, send ratio, and sent latency increase. The input to our network is thus a tensor of shape (B, C, T) where $B =$ batch size $= 64$,

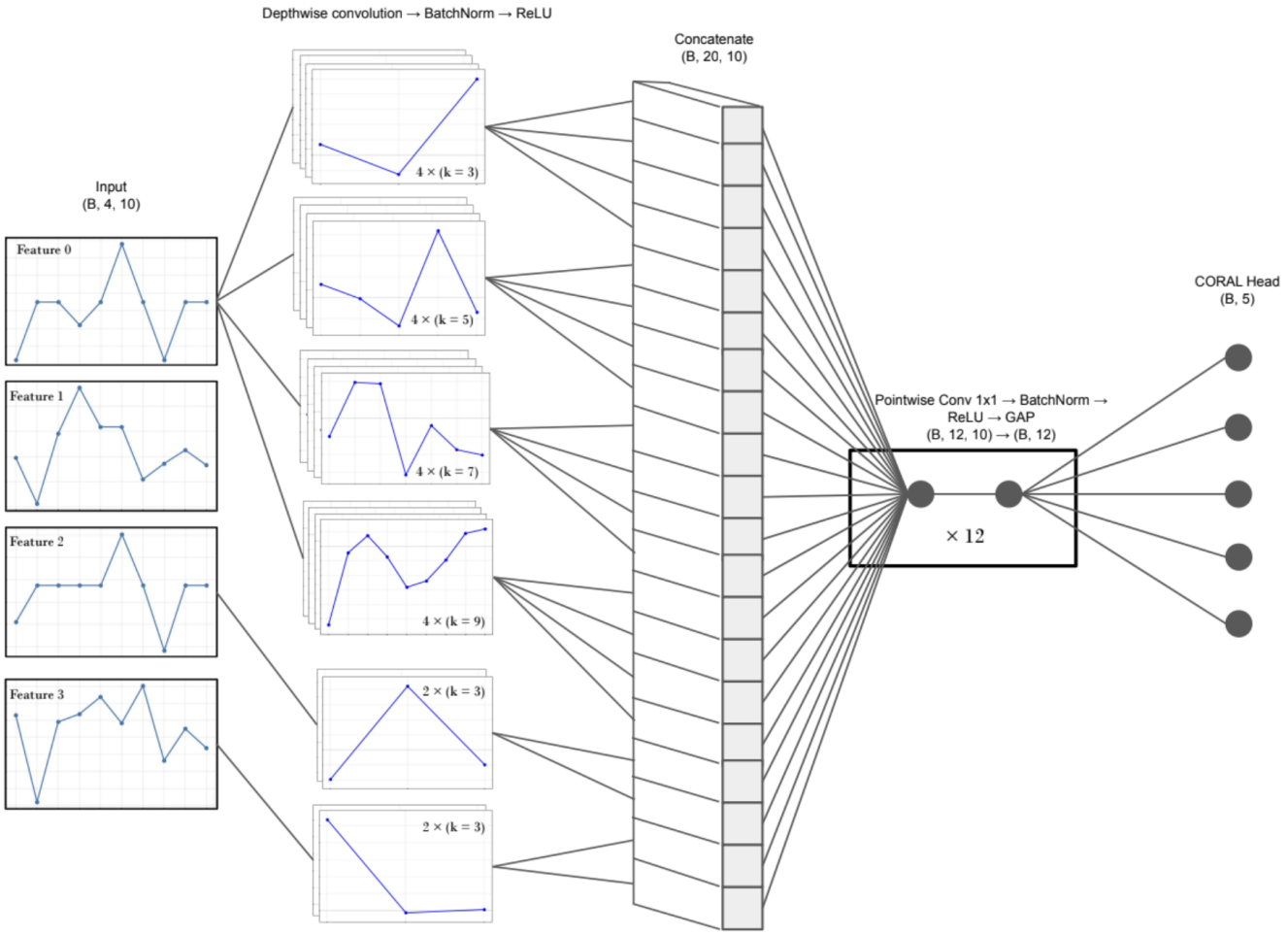


Figure 1: CNN architecture of our Aurora surrogate. The network takes as input 4-channel, 10-timestep multivariate time series data of shape (B, 4, 10) where B is the batch size. Parallel depthwise 1D convolutional filters extract patterns across different temporal scales. The 20 filter outputs are concatenated into a tensor of shape (B, 20, 10), which is then passed to a pointwise convolutional layer. The 12 mixing channels detect combinations of features independently for each time step, yielding an output of shape (B, 12, 10). Global averaging over time produces a tensor of shape (B, 12). The output layer, a 5-neuron CORAL head for ordinal regression, yields a scalar prediction in the range 0 to 5, corresponding to Aurora's 6 actions.

$C =$ #input channels $= 4$, and $T =$ #time steps $= 10$. All input channels were min-max normalized.

The action selected by Aurora for this input serves as our target (see Section "Output Layer" for details).

1. Depthwise Convolutional Branches: Our model uses parallel depthwise convolutional branches with different kernel sizes to capture temporal patterns at multiple scales. To aid in interpretability, each input channel is convolved independently. Initially, all input channels were allocated the same number of kernels. But again motivated by our later goals of interpretability, we reduced the number of filters from 128 to 20.

We decided this allocation scheme by adding to the loss a penalty for using the depthwise filters. A 3-fold cross validation procedure revealed that a large number of the 128 filters in the original network, including all of the kernels allocated to the latency ratio input channel, could be dropped with little effect on the model's accuracy. The loss ratio proved to have an outsized influence on accuracy, and 16 of the 20 filters detect patterns in this input channel.

2. Channel Mixing (Pointwise Convolution) After concatenating outputs from the 20 depthwise filters, 12 mixing channels learn a new set of mixed features. Each of these 12 mixed features is a weighted sum of all the depthwise filter outputs. In contrast to the depthwise filters, which attend to only a single input channel at a time, the mixing channels combine information across channels.

3. Global Average Pooling (GAP) The mixing channels apply a linear transformation of the depthwise features independently at each time step. After batch normalization and ReLU activation, the outputs are averaged across the 10 time steps. GAP makes the model invariant to feature timing within the window, responding only to feature presence.

4. Output Layer Aurora's $K = 6$ actions (i.e., -50%, -33%, -10%, No change, +10%, +33%) have a natural ordering $(0 < 1 < 2 < 3 < 4 < 5)$. We therefore model our surrogate's task as one of ordinal regression rather than multiclass classification. Following the COnsistent RAnk Logits (CORAL) framework [1], we map the 12 outputs of the GAP layer to $K - 1 = 5$ "threshold" logits. Each logit j, after passing through the sigmoid function, represents $P(y > j)$. This cumulative distribution is then converted to a distribution over thresholds, and the network predicts its mode.

During training, each threshold is treated as a binary classification problem with the cross-entropy loss. In situations where the classes have an ordering, a CORAL loss provides better sample efficiency than a standard multilogit loss.

Extracting text descriptions from the CNN's representations: Training a CNN to predict Aurora's output achieves nothing unless its representations are interpretable. We thus prioritized making our architecture amenable to interpretation. The depthwise filters, located in the first layer of the network, can be explained in terms of the inputs. And the mixing channels, in turn, can be explained in terms of the depthwise filters.

To generate natural language descriptions for the 20 depthwise filters, we plotted their weights (Figure 2) and fed those images to GPT 5.1 with Thinking enabled. We prompted the LLM to first "describe the plot in sufficient detail such that the image can be discarded." By feeding the LLM a plot of the weights rather than the weights themselves, we hoped to encourage it to interpret each feature in an abstract, holistic fashion. We then fed the model a detailed prompt specific to the kernel size and feature being explained.

The same protocol was followed for all 20 depthwise filters, yielding a dictionary with filter indices as keys and text descriptions as values. For each of the 12 mixed features, we prompted the LLM to describe a weighted combination of the depthwise filter descriptions, using the mixing channel's learned weights as the coefficients. Each of the LLM's descriptions for the 12 mixed features highlights a distinct network pattern. The output logits of the CORAL head were annotated in light of these mixing channel descriptions.

Feature Attribution Generating a description for a given input $x^{(i)}$ requires a method of determining which depthwise and mixed features were most important to a given prediction $\hat{y}^{(i)}$. Although SHAP values are well suited to this task, we opted for a less expensive, Grad-CAM style method. In particular, we defined the depthwise filters' importance scores as $A_{depth} \times G_{depth}$ where A_{depth} is the concatenated depthwise output and G_{depth} is the gradient of the mode probability with respect to that output. Likewise, we defined the mixing channels' importance scores as $A_{mix} \times G_{mix}$ where A_{mix} is the post-BN/ReLU mixing output and G_{mix} is the gradient of the mode probability with respect to that output. With this approach, computing importance scores for an input $x^{(i)}$ requires only a single backward pass.

5 Experiments

We evaluated all three of our surrogate models on Aurora, a deep-learning controller for congestion control. We also evaluated the nonlinear output mapping—which can be directly substituted for Agua's output mapping—on Gelato (adaptive bitrate streaming) and LUCID (DDoS detection), using the same three DL controllers as in the Agua paper. The data used to train and test the models can be found in the Agua GitHub repository [14].

Following the authors of Agua, we use the fidelity metric FD to assess the quality of our three surrogate models.

$$FD = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\big(f(x^{(i)}) = f'(x^{(i)})\big), \quad \forall x^{(i)} \in \mathcal{D}.$$

In words, the fidelity of a surrogate is the proportion of instances where the surrogate model's predictions match those of the original model on an unseen test dataset $\mathcal{D}$.

5.1 Evaluating Concept Embedding Model

For a fair comparison of the updated CEM AGUA and the baseline CBM model, we use similar model parameters. While the CEM will always be inherently more complex, both the CBM and CEM concept mapping function use the same depth neural network to generate the scalar concepts and concept embeddings respectfully with the same hidden layer width. For the congestion control data, the model uses a single hidden layer with width 100. The output mapping function for both models is a linear model.

As shown in Table 1, fidelity of the CEM model w.r.t. the DL controllers for the Congestion Control dataset were significantly larger than that of the baseline CBM model improving from 0.931 to 0.983 accuracy. While not reported in the table, the CEM validation fidelity on the ABR dataset was 0.957, a significant improvement from the baseline 0.858 validation fidelity. Additionally, we evaluated concept fidelity and found that the CEM model had an improved concept accuracy of 0.695 compared to CBM with a concept accuracy of 0.629. These indicate that the CEM produces more accurate concepts and outputs than the baseline CBM AGUA.

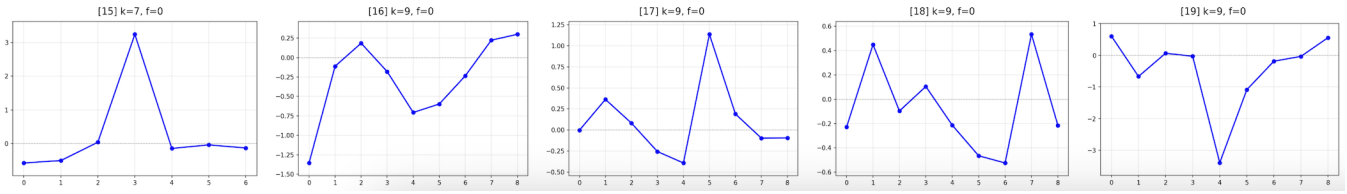


Figure 2: Plots of the weights for 5 of the 20 filters learned by the parallel depthwise branches of the CNN. k denotes the kernel size, or number of consecutive time steps, to which that filter attends. f is the input feature index. These plots were fed to GPT 5.1 one at a time, in separate conversations. In each interaction, it generated a natural language description of the pattern observed. The axes were stripped of titles to encourage the LLM to focus on high-level patterns rather than particular values.

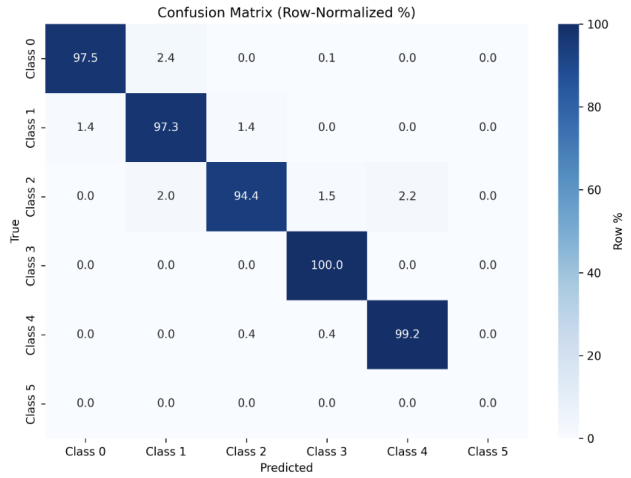


Figure 3: Row-normalized confusion matrix for the CNN-based surrogate on an unseen test dataset. The model has high fidelity to Aurora and exhibits its worse performance when the true action is 2 (i.e., decrease the data transmission rate by 10%). Action 5 was absent from the test set.

5.2 Evaluating Non-Linear Output Mapping

We evaluate whether replacing Agua's ElasticNet-based linear output mapper with our lightweight neural network (Section 4.2) improves the fidelity of the surrogate controller. Fidelity is defined as the accuracy of the surrogate model in predicting the original controller's action on held-out test states.

We compare both approaches across the three domains supported by Agua: Adaptive Bitrate Streaming (ABR), Congestion Control (CC), and Lucid DDoS Detection. The results are reported in Table 1. In all cases, the input to both models is the $(\text{concept}, \text{bin})$ probability vector produced by the concept mapping function; the only difference is the form of the output-mapping head.

Across ABR and Congestion Control, the nonlinear MLP consistently improves fidelity over the linear ElasticNet model. In ABR, fidelity increases from 0.975 to 0.98, and in Congestion Control the improvement is more pronounced, from 0.931 to 0.964. These gains indicate that the relationship between concept activations and controller actions is not purely additive and benefits from modeling nonlinear interactions. For DDoS detection, both models achieve perfect fidelity (1.0), likely because the task is relatively separable in concept space, leaving little room for improvement. Nevertheless, the MLP does not degrade performance, suggesting that the added expressiveness does not harm stability.

Table 1: Output fidelity of Agua and updated models.

Task	**Agua**	**CEM**	**NN Output Mapper**	**CNN-based Explainer**
Adaptive Bitrate	0.975	–	**0.983**	–
Congestion Control	0.931	**0.983**	0.964	0.968
DDoS Detection	1.000	–	1.000	–

5.2.1 Explainability Analysis

Replacing Agua's linear ElasticNet output mapper with a nonlinear MLP improves fidelity (Section 5.2), but introduces nonlinear interactions that are not directly interpretable from model weights alone. To preserve interpretability, we apply a gradient-based saliency method. For each action logit $y = f(x)$ and $(\text{concept}, \text{bin})$ input vector x, we compute

$$\text{saliency}(x_i) = \left| x_i \cdot \frac{\partial y}{\partial x_i} \right|,$$

which quantifies how strongly a small change in input dimension x_i would affect the model's output. Averaging this quantity over all test samples yields a global importance ranking for all concepts and for each bin within a concept.

Because each concept is discretized into three bins (low, medium, high), examining saliency at the bin level is particularly useful. A concept may appear globally important, but the model might only react strongly when the concept is "high" (e.g., severe congestion) or "low" (e.g., nearly empty buffer). Thus, bin-wise saliency reveals how the nonlinear mapper uses graded concept intensities, rather than treating concepts as binary on/off signals. This provides finer-grained insight into controller behavior for example, identifying whether the

MLP responds primarily to extreme conditions or relies on moderate fluctuations. This analysis allows us to answer two interpretability questions:

1. Which concepts (and which intensity levels) most strongly drive the MLP's predictions? This identifies the high-level abstractions that the surrogate model relies on.

2. Does the nonlinear head still base its decisions on meaningful, human-interpretable concepts rather than opaque latent interactions? Since Agua's interpretability hinges on concept-level reasoning, we verify that the MLP's gradients concentrate on real concepts (e.g., "Nearly Full Buffer") rather than arbitrary combinations of bins, ensuring that nonlinearity does not break conceptual alignment.

Figure 4 presents all six saliency plots across the three domains: ABR, Congestion Control, and Lucid DDoS Detection. Each column corresponds to a task. The top row shows the top-K most influential concepts, while the bottom row displays the corresponding bin-wise saliency decompositions, revealing how low, medium, and high intensity activations contribute to the final decision.

From Figures 4(a),(d), the dominant concepts include Nearly Full Buffer, Anticipation of Network Congestion, and High Content Complexity. These align well with canonical ABR decision factors. The per-bin decomposition reveals that the MLP places greatest emphasis on high-intensity bins, suggesting that extreme buffer or congestion conditions drive bitrate changes most strongly.

Figures 4(b),(e) show that the MLP relies on Extreme Network Degradation, Moderate Network Throughput, and Stable Buffer. Interestingly, buffer-related concepts, originally designed for ABR, still emerge as influential, indicating that Agua's shared concept space captures cross-domain structural similarities. The nonlinear head appears to model interactions between throughput variation and queueing behavior.

Figures 4(c),(f) display more diffuse saliency patterns, consistent with the near-perfect separability of the task. While concepts such as Stable Buffer, Recent Improvement in Network, and Rapidly Depleting Buffer still rank highly, their per-bin contributions are more uniform, indicating reliance on broader temporal trends rather than sharp, high-intensity concept activations. Overall, the saliency analysis confirms that the nonlinear mapper remains semantically aligned with Agua's concept space. The MLP leverages meaningful, system-interpretable abstractions rather than spurious correlations, providing both higher fidelity and maintained interpretability.

5.3 Evaluating the CNN-Based Explainer

The CNN-based model modestly improves upon Agua's fidelity to Aurora, increasing it from 0.931 to 0.968. Even when this model predicts an action index incorrectly, it tends to predict an adjacent action (i.e. a similar throughput adjustment) (see Figure 3). We encourage the reader to visit aguarevisited.streamlit.app to see the explainer in action. Explanations may take up to two minutes to generate. We use the surrogate model rather than Aurora for congestion control, yielding a system that implements a reasonable CC policy and explains its own internal decision-making process. Network activity is simulated using the OpenAI Gymnasium environment configured in [12] (see paper [11]).

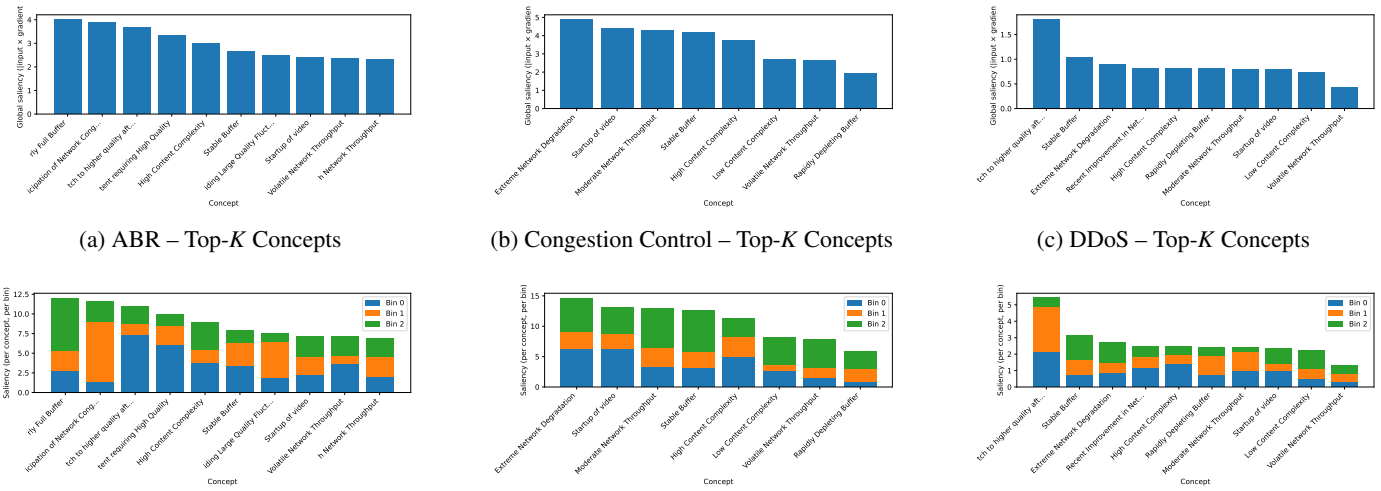
6 Discussion

All three of our methods—the CEM, the NN Output Mapper, and the CNN-based explainer—exhibit higher output fidelity than Agua in the congestion control task. The NN Output Mapper also achieves a similar result in the adaptive bitrate and DDoS detection domains. High fidelity, however, gives insufficient reason to use one of our extensions to Agua. Indeed, the authors of Agua assert that they can achieve higher fidelity by including more base concepts (i.e. expanding the size of the "bottleneck") [13]. So, putting fidelity aside, what do these approaches have to offer?

The NN Output Mapper is best seen as a slight variant of Agua, rather than an entirely new architecture. It offers the advantage of being readily swappable with Agua's linear output mapper, and of not expanding the size of the concept space. Augmented with saliency methods, it provides researchers with a simple way to more accurately approximate a DL controller's output while preserving explainability.

The CEM-based approach makes a more significant modification to Agua's architecture, replacing each "concept" neuron in the bottleneck layer with a sets of neurons (i.e. an embedding). With its more expressive concept space, the CEM can learn nuanced relationships between concepts and better capture what the DL controller is actually using. Given the credibility of CEMs in the XAI community, Agua-CEM presents a natural choice as a successor to the original concept-bottleneck implementation.

The CNN-based approach is the most speculative of the three. While Agua leverages a LLM to generate base concepts *prior to training* the surrogate, this approach leverages a LLM *after training* to annotate the representations learned by the surrogate. In doing so, it lets the model decide what representations are useful for predicting the DL controller's output. Explaining these representations in natural language is difficult but made easier by careful architecture design (e.g. depthwise filters) and by prompting the LLM in a layer-by-layer manner. We intend for this surrogate to serve as a drop-in replacement for the DL controller, but which is capable of explaining (and conversing about) its actions. To further improve the faithfulness of the LLM's explanations to the surrogate's representations, we foresee a training procedure in which the explanations and the representations



(a) ABR – Top-K Concepts

(b) Congestion Control – Top-K Concepts

(c) DDoS – Top-K Concepts

(d) ABR – Bin-wise Decomposition

(e) Congestion Control – Bin-wise Decomposition

(f) DDoS – Bin-wise Decomposition

Figure 4: Saliency analysis of the nonlinear output mapper across the three Agua domains. Each column corresponds to a task. Top: most influential concepts. Bottom: bin-wise saliency decomposition for the same concepts.

are learned concurrently and mapped to the same embedding space.

7 Conclusion

Learning-enabled controllers have been shown to outperform traditional rule-based ones across a range of systems and networking tasks. Yet adoption of these controllers remains low, due in part to their opaque nature. In this paper, we explore three extensions to the Agua framework for concept-based explanations. All three approaches achieve high fidelity at the cost of added complexity. But they differ in their commitment to the original Agua architecture. At present, the CEM-based implementation is the most promising option for operators seeking a more expressive successor to Agua.

References

[1] Wenzhi Cao, Vahid Mirjalili, and Sebastian Raschka. Rank consistent ordinal regression for neural networks with application to age estimation. *Pattern Recognition Letters*, 140:325–331, 2020.

[2] Filip Karlo Došilović, Mario Brčić, and Nikica Hlupić. Explainable artificial intelligence: A survey. In *2018 41st International convention on information and communication technology, electronics and microelectronics (MIPRO)*, pages 0210–0215. IEEE, 2018.

[3] Mateo Espinosa Zarlenga, Pietro Barbiero, Gabriele Ciravegna, Giuseppe Marra, Francesco Giannini, Michelangelo Diligenti, Zohreh Shams, Frederic Precioso, Stefano Melacci, Adrian Weller, et al. Concept embedding models: Beyond the accuracy-explainability trade-off. *Advances in neural information processing systems*, 35:21400–21413, 2022.

[4] Amirata Ghorbani, James Wexler, James Zou, and Been Kim. *Towards automatic concept-based explanations*. Curran Associates Inc., 2019.

[5] Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, and François Petitjean. Inceptiontime: Finding alexnet for time series classification. *arXiv preprint arXiv:1909.04939*, 2019.

[6] Nathan Jay, Noga Rotman, Brighten Godfrey, Michael Schapira, and Aviv Tamar. A deep reinforcement learning perspective on internet congestion control. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.

[7] Been Kim, Justin Gilmer, Martin Wattenberg, Carrie Cai, James Wexler, Fernanda Viegas, and Rory Sayres. Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (tcav). In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018.

[8] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20, 2020.

[9] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS 30)*, 2017.

[10] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the ACM SIGCOMM Conference*, 2017.

[11] Sagar Patel, Sangeetha Abdu Jyothi, and Nina Narodytska. Crystalbox: Future-based explanations for input-driven deep rl systems. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(13):14563–14571, Mar. 2024.

[12] Sagar Patel, Sangeetha Abdu Jyothi, and Nina Narodytska. Crystalbox: Future-based explanations for input-driven deep rl systems – congestion control code. https://github.com/sagar-pa/crystalbox/tree/main, 2024. GitHub repository.

[13] Sagar Patel, Dongsu Han, Nina Narodytska, and Sangeetha Abdu Jyothi. Agua: A concept-based explainer for learning-enabled systems. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*, Coimbra, Portugal, September 2025. Association for Computing Machinery.

[14] Sagar Patel, Dongsu Han, Nina Narodytska, and Sangeetha Abdu Jyothi. Agua: Concept-based explanations for learning-enabled systems – congestion control code. https://github.com/NetSAIL-UCI/agua/tree/main/congestion_control, 2025. GitHub repository.

[15] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. In *Association for Computing Machinery*, 2016.

[16] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence*, 1(5):206–215, 2019.

[17] Simon Schrodi, Julian Schur, Max Argus, and Thomas Brox. Concept bottleneck models without predefined concepts. *arXiv preprint arXiv:2407.03921*, 2024.

[18] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pages 618–626, 2017.

[19] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International conference on machine learning*, pages 3319–3328. PMLR, 2017.

[20] Keith Winstein and Hari Balakrishnan. Tcp ex machina: computer-generated congestion control. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, SIGCOMM '13, page 123–134, 2013.

[21] Yue Yang, Artemis Panagopoulou, Shenghao Zhou, Daniel Jin, Chris Callison-Burch, and Mark Yatskar. Language in a bottle: Language model guided concept bottlenecks for interpretable image classification. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 19187–19197, 2023.

[22] Mert Yuksekgonul, Maggie Wang, and James Zou. Post-hoc concept bottleneck models. *arXiv preprint arXiv:2205.15480*, 2022.